Optimization Foundations for Machine Learning

Machine Learning — Lecture 2

Formatted by: HEISENBERG 😎

This lecture kicks off a short mini-series (2–3 lectures) on **optimization**, the mathematical backbone that makes techniques like SVMs and neural networks work. Before diving into those models, we build up the optimization machinery from scratch — starting with a single-variable refresher and working our way up to vectorized, matrix-based formulations that scale to models with millions of parameters.

Index

- What Is an Optimization Problem? — *the "find the best x" game, formally defined.*
- The Notation Zoo: min, max, argmin, argmax — *four symbols, two meanings, zero excuses for mixing them up.*
- Finding Extrema with Derivatives — *slope-chasing 101, now with worked examples.*
- Worked Example — Minimizing a Parabola — *the friendliest possible function agrees to have exactly one low point.*
- Worked Example — Maximizing an Upside-Down Parabola — *same trick, opposite curvature, same happy ending.*
- When a Function Refuses to Behave: Multiple Extrema — *local minima throwing a party global minima wasn't invited to.*
- Telling Minima from Maxima: The Second Derivative Test — *curvature as a tie-breaker.*
- Leveling Up: Functions of Several Variables — *when one slope becomes a whole vector of slopes.*
- The Gradient Vector — *nabla enters the chat.*
- Vectorizing Everything — *retiring x_1, x_2, x_3 … in favor of one tidy vector x.*
- Two Gradient Rules Worth Memorizing — *the only two identities you'll reuse for the rest of the course.*
- The General Quadratic Form and Its Gradient — *matrices join the party.*
- Putting It to Work: Deriving Linear Regression from First Principles — *watch a loss function turn into the normal equation before your eyes.*
- Practice Questions — *five questions to make sure this actually stuck.*

What Is an Optimization Problem?

At its core, optimization is about finding the input to a function that makes the function's output as large or as small as possible. Most of this material was already touched on in high school (11th/12th grade calculus), so the goal here is to refresh it quickly before machine-learning-specific complications get piled on top.

Start with the simplest possible setting: a function $f : \mathrm{R} \to \mathrm{R}$, meaning it takes a single real number as input and produces a single real number as output. A concrete example used throughout this lecture is:

$$f(x) = x^2 + 2x + 3, \qquad x \in \mathrm{R}$$

Both the input and the output here are **scalars** — single numbers, not vectors or matrices.

In machine learning, the overwhelming majority of problems are framed as **minimization** problems (minimizing a loss or cost function), even though the underlying mathematics works identically for maximization.

The Notation Zoo: min, max, argmin, argmax

Two closely related but distinct ideas need separate notation:

- $f_{\max} = \max_x f(x)$ — the **maximum value** that the function attains.
- $f_{\min} = \min_x f(x)$ — the **minimum value** that the function attains.
- $x^* = \arg\max_x f(x)$ — the **location** (the value of x) at which the maximum occurs.
- $x^* = \arg\min_x f(x)$ — the **location** at which the minimum occurs.

> **Note for Review:** This mix-up is a classic exam trap. `min f(x)` answers "how low does it go?" while `argmin f(x)` answers "where does it go that low?" One returns a function *value*, the other returns a *point*.

Finding Extrema with Derivatives

The standard recipe for locating a maximum or minimum of a single-variable function is to look for points where the function's **slope is zero**.

Why does this work? Consider approaching a point of minimum from the left:

Region relative to the minimum	Sign of $f'(x)$
Just before the minimum	Negative (function is decreasing)
Exactly at the minimum	Zero
Just after the minimum	Positive (function is increasing)

The slope transitions from negative to positive, passing through zero exactly at the minimum. The mirror image holds for a maximum:

Region relative to the maximum	Sign of $f'(x)$
Just before the maximum	Positive (function is increasing)
Exactly at the maximum	Zero
Just after the maximum	Negative (function is decreasing)

This gives the general recipe: **differentiate the function, set the derivative to zero, and solve for x.**

Worked Example — Minimizing a Parabola

Take $f(x) = x^2 + 2x + 3$ and find its minimum.

Step 1 — Differentiate:

$$\frac{df(x)}{dx} = 2x + 2$$

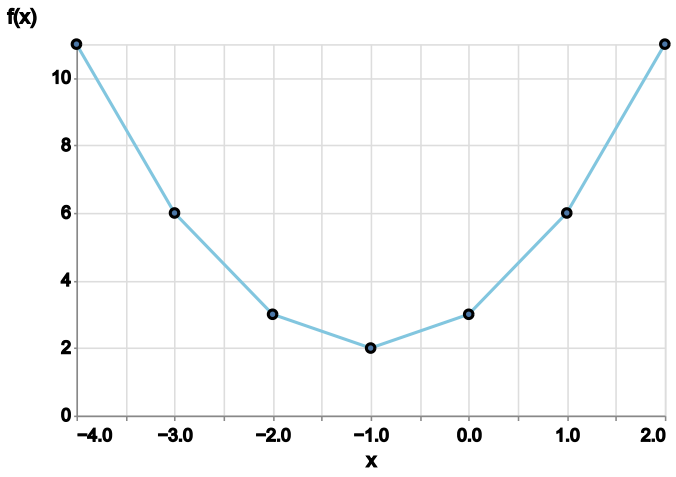
Step 2 — Set the derivative to zero and solve:

$$2x + 2 = 0 \implies x^* = -1$$

Step 3 — Evaluate the function at the optimal point:

$$f(x^*) = (-1)^2 + 2(-1) + 3 = 1 - 2 + 3 = 2$$

So the function attains its minimum value of 2 at $x^* = -1$. Before $x = -1$ the slope is negative; after it, the slope is positive — confirming this is indeed a minimum, not just a stationary point.



Worked Example — Maximizing an Upside-Down Parabola

Now take $f(x) = -x^2 + 2x + 1$ and find its **maximum**.

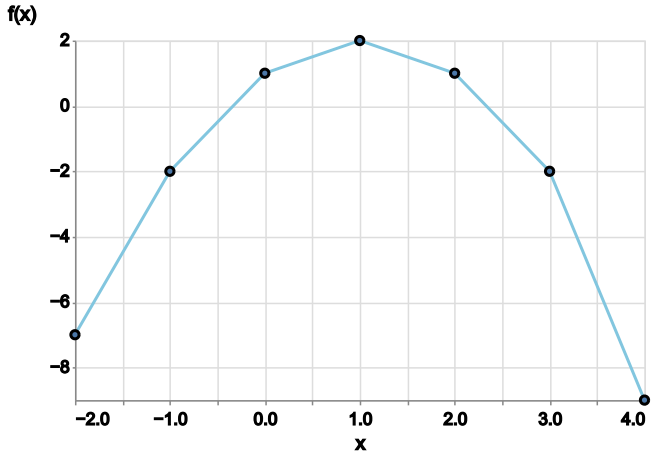
Step 1 — Differentiate:

$$\frac{df(x)}{dx} = -2x + 2$$

Step 2 — Set the derivative to zero and solve:

$$-2x + 2 = 0 \implies x^* = 1$$

The function increases before $x = 1$ and decreases after it, confirming a maximum at $x^* = 1$, with $f(1) = -1 + 2 + 1 = 2$.

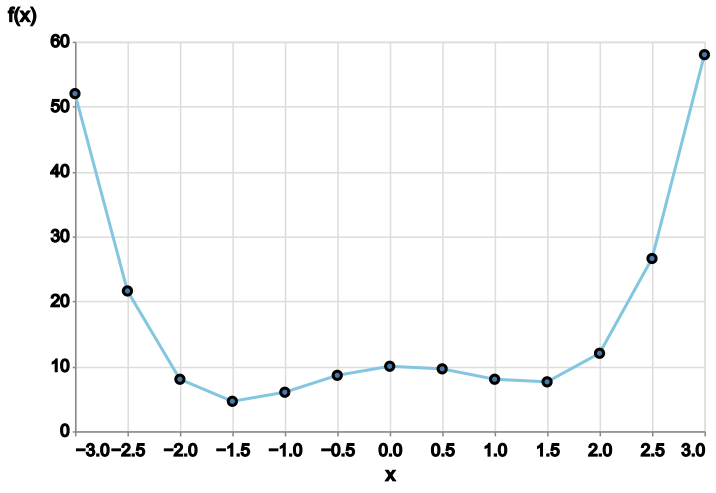


When a Function Refuses to Behave: Multiple Extrema

Both worked examples above are unusually cooperative: each has **exactly one** stationary point, so setting the derivative to zero immediately identifies the global optimum. Real functions — and in particular, the loss surfaces of neural networks — are rarely this polite. A function can easily have **multiple local minima and multiple local maxima**, each corresponding to a point of zero slope.

The recipe still starts the same way — find every point where $f'(x) = 0$ — but now there is an extra step:

1. Find **all** stationary points (every x where $f'(x) = 0$).
2. Evaluate $f(x)$ at each one.
3. The stationary point with the **smallest** function value is the **global minimum**; the one with the **largest** value is the **global maximum**. All the others are merely *local* extrema.



Reading the curve left to right: the slope is negative, hits zero at the first (local) minimum, turns positive, hits zero again at a local maximum, turns negative, hits zero at a second minimum, and finally turns positive again on the far right. Every one of those zero-crossings is a stationary point, but only the deepest dip qualifies as the **global** minimum.

Telling Minima from Maxima: The Second Derivative Test

Once a stationary point is found, how do we know whether it's a minimum or a maximum without plotting the whole curve? The **second derivative** (curvature) settles it:

Second derivative at x^*	Interpretation	Type of extremum
$f''(x^*) > 0$	Curving upward (like a bowl)	**Minimum**
$f''(x^*) < 0$	Curving downward (like a dome)	**Maximum**

Checking the two worked examples: for $f(x) = x^2 + 2x + 3, f''(x) = 2 > 0$, confirming a minimum; for $f(x) = -x^2 + 2x + 1, f''(x) = -2 < 0$, confirming a maximum.

Leveling Up: Functions of Several Variables

Real machine learning models rarely depend on a single scalar input — a data point might have hundreds of features, and a model might have thousands (or millions) of parameters. So the next step is to generalize from $f : \mathrm{R} \to \mathrm{R}$ to functions that take a **vector** as input:

$$f : \mathrm{R}^2 \to \mathrm{R}, \qquad f(x_1, x_2) = x_1^2 + x_2^2 + 2$$

Here x_1 and x_2 are stacked into a vector $x = [\begin{matrix} x_1 \\ x_2 \end{matrix}]$. Since $x_1^2 + x_2^2$ is exactly the squared 2-norm of x, this function can be rewritten compactly as:

$$f(x) = x^\top x + 2$$

With a single-variable function there was only one direction in which to measure slope. With two (or more) variables, there is a slope **in each coordinate direction**, captured by **partial derivatives**:

$$\frac{\partial f}{\partial x_1} = 2x_1, \qquad \frac{\partial f}{\partial x_2} = 2x_2$$

Setting both partial derivatives to zero simultaneously gives the stationary point:

$$\frac{\partial f}{\partial x_1} = 0 \implies x_1^* = 0, \qquad \frac{\partial f}{\partial x_2} = 0 \implies x_2^* = 0$$

Checking curvature in each direction, $\frac{\partial^2 f}{\partial x_1^2} = 2 > 0$ and $\frac{\partial^2 f}{\partial x_2^2} = 2 > 0$, so this is a point of minima.

The same idea extends to three variables, $f : \mathrm{R}^3 \to \mathrm{R}$:

$$f(x_1, x_2, x_3) = x_1^2 + x_2^2 + x_3^2 + 3$$

...and beyond. In a large neural network, the loss function $f(\theta)$ might depend on a parameter vector $\theta \in \mathrm{R}^n$ where n runs into the thousands or millions:

$$\min_\theta f(\theta), \qquad \theta \in \mathrm{R}^n$$

Not every multivariable function separates as cleanly as the example above. Consider:

$$f(x_1, x_2) = x_1^2 + 2x_2^2 + 2x_1x_2 + 3$$

Here the partial derivatives are:

$$\frac{\partial f}{\partial x_1} = 2x_1 + 2x_2 = 0 \implies x_1 + x_2 = 0$$

$$\frac{\partial f}{\partial x_2} = 4x_2 + 2x_1 = 0 \implies x_1 + 2x_2 = 0$$

Unlike the earlier example, both equations now involve **both** variables — they must be solved **simultaneously**. This is still a manageable pair of linear equations, but the difficulty escalates quickly. For instance, with

$$f(x_1, x_2) = 2x_1^4 + 3x_2^3 + 2x_1x_2 + 6$$

the resulting stationary-point equations,

$$8x_1^3 + 2x_2 = 0, \qquad 9x_2^2 + 2x_1 = 0$$

are **non-linear**. Solving a handful of non-linear equations is already unpleasant; solving thousands of them — as would be required for a real neural network — is intractable by hand. This is precisely why practical machine learning relies on **numerical optimization algorithms** (like gradient descent) rather than directly solving the stationary-point equations, a topic for a later lecture.

The Gradient Vector

Rather than juggling every partial derivative separately, they are collected into a single vector called the **gradient**, denoted ∇f:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix}$$

For a function of n variables, the gradient is an $n \times 1$ vector containing all n partial derivatives. Finding a stationary point of a multivariable function now becomes a single, compact statement:

$$\nabla f = 0$$

Vectorizing Everything

Writing out $x_1, x_2, x_3, \ldots$ individually becomes unmanageable once a model has many parameters. The fix is to stack all the variables into a single vector x and re-express both the function and its gradient purely in terms of x .

Take $f(x_1, x_2) = x_1^2 + x_2^2 + 2x_1 + 3x_2 + 6$. Grouping the linear coefficients into a vector $b = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$, this becomes:

$$f(x) = x^\top x + b^\top x + 6$$

Its gradient, found term by term, is:

$$\nabla f = 2x + b$$

This vectorized form is the version that will be reused for the rest of the course — occasionally we'll drop back down to individual partial derivatives to explain a new algorithm, but the endgame is always to express both the function and its gradient in terms of vectors.

Two Gradient Rules Worth Memorizing

Comparing the vectorized function to its gradient reveals three identities that recur constantly throughout machine learning:

Term	Gradient with respect to x
$x^\top x$	$2x$
$b^\top x$ (constant vector b)	b
constant c	0

> **Note for Review:** These three rules are the "multiplication table" of ML optimization — nearly every gradient derivation in this course (linear regression, logistic regression, ridge regression, and beyond) boils down to applying these three identities to a rewritten loss function.

The General Quadratic Form and Its Gradient

The next level of generality introduces a **matrix** into the function. Let A be a 2×2 matrix, b a 2×1 vector, and c a scalar constant:

$$A = \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$f(x) = x^\top Ax + b^\top x + c$$

Dimensionally, $x^\top Ax$ is $(1 \times 2)(2 \times 2)(2 \times 1) = 1 \times 1$, so the whole expression is a scalar, exactly as required.

Deriving the gradient of $x^\top Ax$: Expanding the product explicitly,

$$x^\top Ax = a_1 x_1^2 + a_3 x_1 x_2 + a_2 x_1 x_2 + a_4 x_2^2$$

Differentiating with respect to each variable:

$$\frac{\partial (x^\top Ax)}{\partial x_1} = 2a_1 x_1 + a_3 x_2 + a_2 x_2, \qquad \frac{\partial (x^\top Ax)}{\partial x_2} = a_3 x_1 + a_2 x_1 + 2a_4 x_2$$

Splitting each partial derivative into two pieces and regrouping by matrix, these two equations recombine into:

$$\nabla_x (x^\top Ax) = Ax + A^\top x = (A + A^\top)x$$

If A is **symmetric** (i.e. $A^\top = A$, which happens often in ML), this simplifies further to:

$$\nabla_x (x^\top Ax) = 2Ax$$

Gradient of the linear term: Expanding $b^\top x = b_1 x_1 + b_2 x_2$ and differentiating gives exactly b back:

$$\nabla_x (b^\top x) = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = b$$

Putting it together:

$$\nabla_x f = (A + A^\top)x + b = 2Ax + b \quad \text{when } A^\top = A$$

This result holds regardless of how large x is — whether it has 2 entries or a million, the gradient of $x^\top Ax + b^\top x + c$ is always $(A + A^\top)x + b$.

Putting It to Work: Deriving Linear Regression from First Principles

With the gradient toolkit built, it can now be applied to a familiar model: **linear regression**.

Suppose the dataset has n points, each with two features, and a target (label) value:

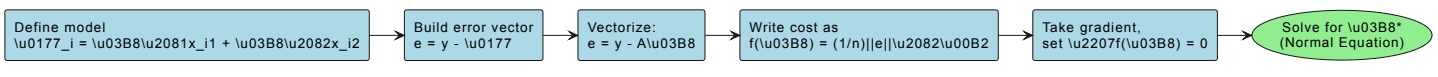
$$(x_{11}, x_{12}, y_1),\ (x_{21}, x_{22}, y_2),\ \ldots,\ (x_{n1}, x_{n2}, y_n)$$

For any data point $x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}$, the linear regression model predicts:

$$\hat{y}_i = h_\theta(x_i) = \theta_1 x_{i1} + \theta_2 x_{i2}, \qquad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$

The learning goal is to choose θ so that every prediction $\hat{y}_i$ is as close as possible to its true label y_i. This is captured by the **mean squared error** cost function:

$$f(\theta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \qquad \theta^* = \arg\min_\theta f(\theta)$$



Step 1 — Express the error as a vector. Stack all n residuals into a single error vector:

$$e = \begin{bmatrix} y_1 - \hat{y}_1 \\ y_2 - \hat{y}_2 \\ \vdots \\ y_n - \hat{y}_n \end{bmatrix}$$

Since $\|x\|_2^2 = \sum_{i=1}^{n} x_i^2$, the cost function is exactly the squared 2-norm of this error vector:

$$f(\theta) = \frac{1}{n}\|e\|_2^2$$

Step 2 — Separate θ from the data. Writing out each predicted value $\hat{y}_i = \theta_1 x_{i1} + \theta_2 x_{i2}$, the error vector splits into a data matrix A (containing every feature) multiplied by θ, subtracted from y:

$$e = y - A\theta, \qquad A = \begin{bmatrix} x_{11} & x_{12} \\ \vdots & \vdots \\ x_{n1} & x_{n2} \end{bmatrix}$$

Step 3 — Expand into the quadratic form. Using $\|x\|_2^2 = x^\top x$:

$$f(\theta) = \frac{1}{n}(y - A\theta)^\top (y - A\theta) = \frac{1}{n}\left(y^\top y - y^\top A\theta - \theta^\top A^\top y + \theta^\top A^\top A\theta\right)$$

Since $y^\top A\theta$ and $\theta^\top A^\top y$ are both scalars and are transposes of one another, they must be equal, so the two middle terms combine:

$$f(\theta) = \frac{1}{n}\left(\theta^\top A^\top A\theta - 2y^\top A\theta + y^\top y\right)$$

Step 4 — Recognize the general quadratic form. Defining $B = A^\top A$, $b = A^\top y$, and $c = y^\top y$:

$$f(\theta) = \frac{1}{n}\left(\theta^\top B\theta - 2b^\top\theta + c\right)$$

This is exactly the quadratic form worked out earlier, with B playing the role of A (and B is automatically symmetric, since $(A^\top A)^\top = A^\top A$).

Step 5 — Differentiate and solve. Applying the gradient rule derived above (with the symmetric-matrix shortcut):

$$\nabla_\theta f(\theta) = \frac{1}{n}(2B\theta - 2b) = 0$$

$$B\theta = b \implies \theta = B^{-1}b$$

Substituting back $B = A^\top A$ and $b = A^\top y$ gives the celebrated **normal equation** of linear regression:

$$\boxed{\theta^* = (A^\top A)^{-1}A^\top y}$$

This single closed-form expression gives the optimal regression coefficients directly — no iterative search required for this particular model. The overall pattern used to get here (write the loss in vector form → find its gradient → set the gradient to the zero vector → solve) is the same pattern that will reappear, in progressively more complex forms, throughout the rest of the course.

Practice Questions

1. For the two-minima curve discussed above, setting $f'(x) = 0$ produces *three* candidate points, not two. Explain what role the third point plays, and how you would prove — using only calculus tools introduced in this lecture — that it is not itself a minimum or a global maximum.

2. The normal equation $\theta^* = (A^\top A)^{-1} A^\top y$ requires $A^\top A$ to be invertible. Think about what it would mean, in terms of the dataset itself (rows and columns of A), for $A^\top A$ to *fail* to be invertible, and why that would break this closed-form approach.

3. Suppose a loss function is given by $f(\theta) = \theta^\top A \theta$ where A is **not** symmetric. Walk through why the "shortcut" formula $\nabla f = 2A\theta$ would give a wrong answer here, and what you would have to compute instead.

4. The lecture argues that for functions with millions of parameters, solving $\nabla f(\theta) = 0$ directly becomes intractable, motivating iterative numerical methods instead of the closed-form approach used for linear regression. Given that linear regression itself *can* have thousands of features, propose a reason why the closed-form normal equation is still usually feasible there, while a similarly sized neural network loss is not.

5. Consider a cost function shaped like a long, shallow valley with steep walls on either side rather than a clean bowl. Reasoning only from the first- and second-derivative tests covered in this lecture (without invoking gradient descent mechanics), what practical difficulty would this shape create if you tried to numerically search for the point where the gradient is exactly zero?